

Лабораторная работа № 7

Тема: Создание учебного CA и выпуск TLS-сертификата для веб-сервера

Цель: Приобрести практические навыки создания собственного удостоверяющего центра, выпуска TLS-сертификата с SAN, настройки HTTPS на веб-сервере и проверки цепочки доверия с клиента.

Ориентировочное время: 4 академических часа

Среда: VirtualBox

ОС: Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS, клиент Debian/Ubuntu или Windows 10/11

1. Необходимые ресурсы

- `ca01` - учебный сервер CA, внутренняя сеть `tls-lab`, адрес `192.168.10.70/24`;
- `web01` - веб-сервер, внутренняя сеть `tls-lab`, адрес `192.168.10.20/24`;
- `client01` - клиент для проверки доверия, адрес `192.168.10.101/24`;
- пакеты `openssl`, `nginx`, `curl`, `ca-certificates`;
- доступ серверов к репозиториям через NAT или заранее подготовленные пакеты;
- работающее разрешение имени `web01.company.test` через DNS или запись в `/etc/hosts`;
- методичка `methodical_guide.md`.

2. Профессиональная ситуация

В организации появились внутренние веб-службы, которые передают административные данные по HTTP. Нужно создать учебный корневой СА, выпустить сертификат для `web01.company.test`, настроить HTTPS и установить доверие к СА на клиенте. Пользователь должен открыть веб-сервер без предупреждения о неизвестном издателе.

Учебный СА используется только в лабораторной сети. Его ключи нельзя применять в реальной инфраструктуре.

3. Исходные данные и ограничения

Параметр	Значение
Внутренняя сеть VirtualBox	<code>tls-lab</code>
Учебный СА	<code>ca01, 192.168.10.70/24</code>
Web-сервер	<code>web01, 192.168.10.20/24</code>
Клиент	<code>client01, 192.168.10.101/24</code>
Имя сервера	<code>web01.company.test</code>
Корневой сертификат СА	<code>Company Test Root CA</code>

HTTPS-порт	443/tcp
HTTP-порт для исходной проверки	80/tcp
SAN сертификата	DNS:web01.company.test, IP:192.168.10.20
Срок серверного сертификата	365 дней

Если DNS-зона `company.test` не настроена, добавьте временную запись имени на клиенте. В отчёте укажите, каким способом имя `web01.company.test` разрешается в IP-адрес.

4. Требования безопасности

Важно: закрытый ключ СА является самым чувствительным файлом стенда. Не копируйте его на клиент, не прикладывайте к отчёту и не храните в общей папке.

Соблюдайте требования:

- перед созданием СА сделайте снимок `ca01`;
- защищайте закрытый ключ СА правами `400` или строже;
- защищайте закрытый ключ web-сервера правами `600` или строже;
- перед перезагрузкой Nginx выполняйте `nginx -t`;
- не прикладывайте в отчёт содержимое файлов `*.key`;

- проверяйте SAN до установки сертификата на веб-сервер.

5. Порядок выполнения работы

Этап 1. Подготовьте стенд и проверку имени

Проверьте IP-адреса ВМ, доступность `web01` с клиента и работу HTTP на веб-сервере. Настройте разрешение имени `web01.company.test` через DNS или локальный файл `hosts`.

Контрольные точки:

```
curl -I http://192.168.10.20
getent hosts web01.company.test
```

Критерий перехода: клиент достигает web-сервера по IP, а имя `web01.company.test` разрешается предсказуемым способом.

Этап 2. Создайте структуру учебного СА

На `ca01` подготовьте отдельный каталог СА с подкаталогами для закрытого ключа, сертификатов и CSR. Ограничьте доступ к каталогу с закрытым ключом.

Контрольная точка:

```
ls -ld /root/ca /root/ca/private /root/ca/certs /root/ca/csr
```

Критерий перехода: структура СА создана, каталог `private` недоступен обычным пользователям.

Этап 3. Создайте закрытый ключ и корневой сертификат СА

Создайте закрытый ключ CA с парольной фразой и самоподписанный корневой сертификат `Company Test Root CA`.

Контрольные точки:

```
openssl x509 -in /root/ca/certs/ca.crt -text -noout  
openssl x509 -in /root/ca/certs/ca.crt -noout -subject -issuer -dates
```

Критерий перехода: сертификат CA самоподписан, имеет корректный subject и срок действия.

Этап 4. Создайте ключ и CSR для `web01`

На `web01` создайте закрытый ключ сервера и CSR для `web01.company.test`.

Подготовьте файл расширений, в котором явно указаны SAN

DNS:`web01.company.test` и IP:`192.168.10.20`.

Контрольная точка:

```
openssl req -in web01.csr -noout -subject -text
```

Критерий перехода: CSR относится к `web01.company.test`, закрытый ключ не покидает web-сервер без необходимости.

Этап 5. Подпишите CSR на CA

Передайте CSR на `ca01` и выпустите серверный сертификат `web01.crt`.

Проверьте issuer, subject, срок действия и SAN.

Контрольные точки:

```
openssl x509 -in web01.crt -text -noout  
openssl verify -CAfile ca.crt web01.crt
```

Критерий перехода: проверка возвращает `web01.crt`: ОК, в SAN есть DNS-имя и IP-адрес web-сервера.

Этап 6. Настройте HTTPS на Nginx

Разместите сертификат и закрытый ключ на `web01` в системных каталогах.

Настройте виртуальный сервер Nginx для HTTPS на `443/tcp`.

В конфигурации должны быть указаны:

- `listen 443 ssl;`
- `server_name web01.company.test;`
- путь к сертификату `web01.crt`;
- путь к закрытому ключу `web01.key`;
- современные версии TLS.

Контрольные точки:

```
sudo nginx -t
sudo systemctl reload nginx
sudo ss -tulpen | grep ':443'
```

Критерий перехода: Nginx принимает HTTPS-подключения на порту `443`.

Этап 7. Импортируйте CA на клиент и проверьте доверие

Установите корневой сертификат CA в доверенное хранилище `client01`.

Проверьте HTTPS через `curl`, браузер или `openssl s_client`.

Контрольные точки:

```
curl --cacert ca.crt https://web01.company.test
openssl s_client -connect web01.company.test:443 -CAfile ca.crt -servername
web01.company.test
```

Критерий перехода: проверка TLS завершается без ошибки доверия, `Verify return code` равен 0 (ok).

Этап 8. Выполните диагностику ошибки TLS

Смоделируйте одну безопасную ошибку: обращение к серверу по имени, отсутствующему в SAN, отсутствие импортированного CA на клиенте или неправильные права на ключ. Найдите причину и восстановите рабочее состояние.

Контрольная точка: в отчёте есть ошибка, диагностическая команда и исправление.

6. Итоговая проверка

Убедитесь, что:

- CA имеет защищённый закрытый ключ и корневой сертификат;
- серверный сертификат выпущен для `web01.company.test`;
- SAN содержит `DNS:web01.company.test` и `IP:192.168.10.20`;
- `openssl verify` подтверждает цепочку доверия;
- Nginx слушает `443/tcp`;
- клиент доверяет CA;
- HTTPS открывается без предупреждения о неизвестном издателе;
- диагностическое действие выполнено.

7. Паспорт TLS-сервиса

Параметр	Значение на стенде
Имя CA	Company Test Root CA
Сервер CA	
Web-сервер	web01.company.test
IP web-сервера	192.168.10.20
Subject серверного сертификата	
SAN сертификата	
Issuer сертификата	
Срок действия сертификата	
Путь к сертификату Nginx	
Путь к закрытому ключу Nginx	
Команда проверки цепочки	
Команда сетевой TLS-проверки	

8. Что приложить к отчёту

Приложите 6-8 доказательств:

1. Схему стенда с `ca01`, `web01` и `client01`.
2. Вывод `subject/issuer/dates` корневого CA.
3. Вывод CSR или сертификата, подтверждающий имя `web01.company.test`.
4. Фрагмент сертификата с SAN.
5. Вывод `openssl verify -CAfile ca.crt web01.crt`.
6. Фрагмент HTTPS-конфигурации Nginx без закрытого ключа.
7. Вывод `curl` или `openssl s_client` с успешной проверкой доверия.
8. Описание одной TLS-ошибки и способа её диагностики.

Не прикладывайте содержимое файлов `ca.key`, `web01.key`, пароли и приватные ключи.

9. Критерии успешного выполнения

Работа выполнена успешно, если:

- создан учебный CA;
- серверный сертификат выпущен через CSR;
- SAN соответствует способу подключения клиента;
- Nginx настроен на HTTPS;
- клиент доверяет корневому CA;
- TLS-проверка проходит без ошибки имени и издателя;
- закрытые ключи защищены правами доступа;

- отчёт содержит доказательства без секретных данных;
- студент может объяснить CA, CSR, SAN, закрытый ключ и цепочку доверия.

10. Контрольные вопросы

9. Какие угрозы возникают при передаче HTTP, LDAP или почтового трафика без TLS?
10. Чем закрытый ключ отличается от сертификата?
11. Почему CSR можно передавать CA, а закрытый ключ нельзя?
12. Какую роль выполняет CA в цепочке доверия?
13. Почему современный сертификат должен содержать SAN?
14. Что проверяет команда `openssl verify`?
15. Что означает ошибка неизвестного издателя сертификата?
16. Что проверить при ошибке несоответствия имени сертификата?